

**Indian Institute of Technology Bombay**

Department of Computer Science and Engineering

**CS683**

Advanced Computer Architecture

---

**Programming Assignment 1**

Dhurandhar Programming

---

**Task 2: SGEMM Optimization**

**Team Runtime Terrors**

**Member 1:** Deepika Kale – 24b1203

**Member 2:** Harshvardhan Patil – 24b1205

**Member 3:** Urjith Karthikeya – 24b1308

**Member 4:** Druthi Sree Somineni – 24b1252

Autumn 2026

## 1 Introduction

This work investigates the performance optimization of single-precision general matrix multiplication (SGEMM) for inference-oriented workloads. The supplied implementation was retained as the correctness reference and performance baseline. The optimization was carried out using explicit SIMD, register blocking, cache blocking, and software prefetching, followed by a combined implementation and llama.cpp integration.

## Experimental Setup

All experiments were performed on a 13th Gen Intel Core i9-13900HX processor. The processor supports SSE, AVX, AVX2 and FMA instructions; AVX2 was therefore the widest SIMD instruction set used in this study. The relevant cache hierarchy consists of a 48 KB L1 data cache, a 2 MB private L2 cache and a 36 MB shared LLC.

**Table 1:** Experimental configuration.

| Parameter     | Configuration                    |
|---------------|----------------------------------|
| CPU           | Intel Core i9-13900HX (13th Gen) |
| Architecture  | x86-64                           |
| L1 Data Cache | 48 KB                            |
| L2 Cache      | 2 MB per P-core                  |
| LLC           | 36 MB shared                     |
| SIMD          | SSE / AVX2 + FMA                 |
| Compiler      | GCC, C++17                       |
| Optimization  | -O2                              |
| Vectorization | -fno-tree-vectorize              |
| SIMD flags    | -mavx2 -mfma                     |
| Execution     | Single-threaded                  |

The matrix multiplication follows

$$C_{ij} = \sum_{p=0}^{K-1} A_{ip} B_{jp}, \quad (1)$$

with the contraction dimension contiguous in both operands. The computational cost is therefore

$$F = 2MNK. \quad (2)$$

Performance is evaluated using execution time, GFLOP/s and speedup over the supplied reference implementation. Correctness is verified against the reference kernel after each optimization stage.

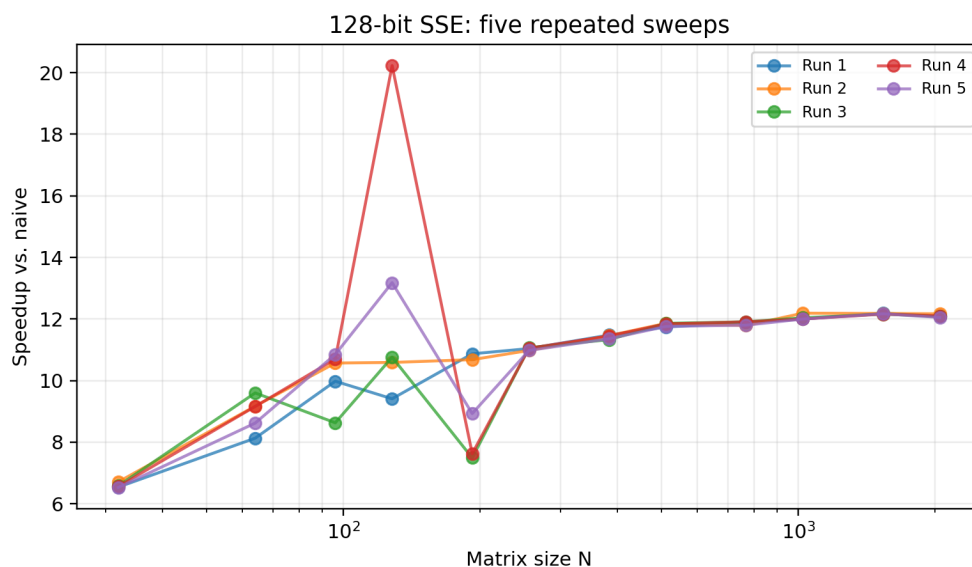
The study is organized around three stages. SIMD and register-tile parameters are first explored to increase arithmetic throughput. Software prefetching is then investigated through prefetch distance, cache-fill hints, cache-panel size and hardware-prefetcher state. Finally, the best-performing configurations are combined and compared against the individual optimizations.

*The optimization strategy was driven by measurement rather than assumed performance behavior.*

## 2 Task 2B — SIMD Optimization

### 2.1 128-bit SIMD: first vectorization step

We first implemented a 128-bit SSE version. A 128-bit register holds four FP32 values, so the inner dot product can process four elements per vector operation. Five independent sweeps were used to observe both the performance trend and run-to-run variation.

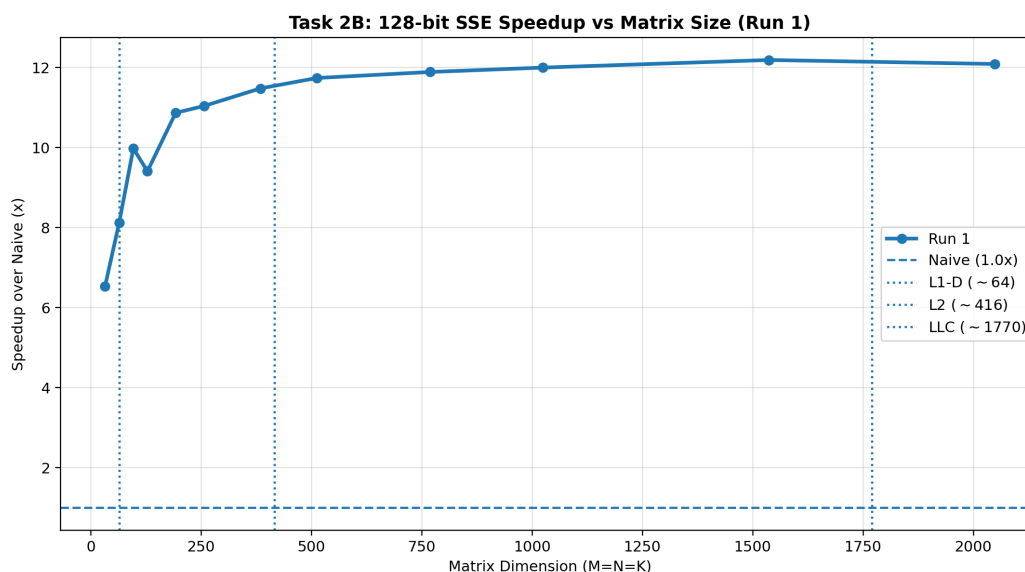


**Figure 1:** Five independent 128-bit SSE sweeps across matrix size. Variability is largest in the small and transition regions, while the large- $N$  behavior becomes stable.

**Table 2:** Median 128-bit SSE results over five trials.

| $N$  | Speedup | Std. dev. | GFLOP/s |
|------|---------|-----------|---------|
| 32   | 6.56    | 0.07      | 25.35   |
| 64   | 9.16    | 0.57      | 22.74   |
| 96   | 10.57   | 0.91      | 28.42   |
| 128  | 10.76   | 4.36      | 47.33   |
| 192  | 8.93    | 1.61      | 37.48   |
| 256  | 11.04   | 0.04      | 47.99   |
| 384  | 11.45   | 0.07      | 48.05   |
| 512  | 11.84   | 0.06      | 48.13   |
| 1024 | 12.00   | 0.08      | 47.04   |
| 1536 | 12.17   | 0.01      | 47.18   |
| 2048 | 12.09   | 0.04      | 46.35   |

The SSE kernel reaches about  $12\times$  speedup for large matrices, with throughput settling near 47 GFLOP/s. This gave us a useful baseline for testing whether a wider vector could increase arithmetic throughput further.

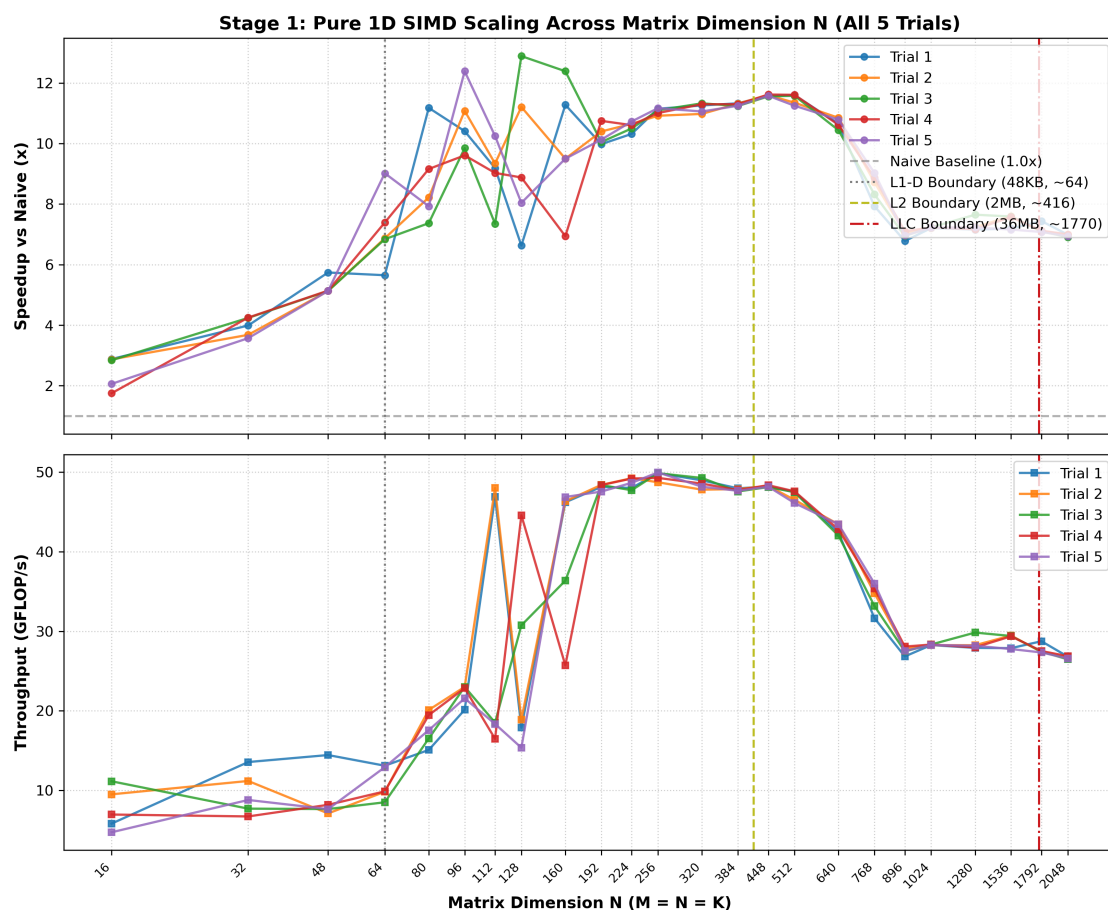


**Figure 2:** Representative 128-bit SSE run with approximate aggregate cache-capacity markers. The markers are working-set guides, not exact cache-transition points; the measured speedup flattens once the kernel reaches its steady-state regime.

For square matrices, the aggregate size of  $A$ ,  $B$ , and  $C$  is approximately  $12N^2$  bytes. Using the measured 48 KB L1-D, 2 MB L2, and 36 MB LLC capacities gives rough size markers of  $N \approx 64$ ,  $N \approx 416$ , and  $N \approx 1770$ , respectively. These values are used only as visual guides because cache associativity, replacement, and simultaneous live data prevent a hard boundary interpretation.

## 2.2 256-bit AVX2

The target processor supports AVX2 and FMA. A 256-bit YMM register holds eight FP32 values, doubling the vector width of SSE. We repeated the same matrix-size sweep using the AVX2 implementation, with five independent trials to distinguish steady-state behavior from measurement noise.



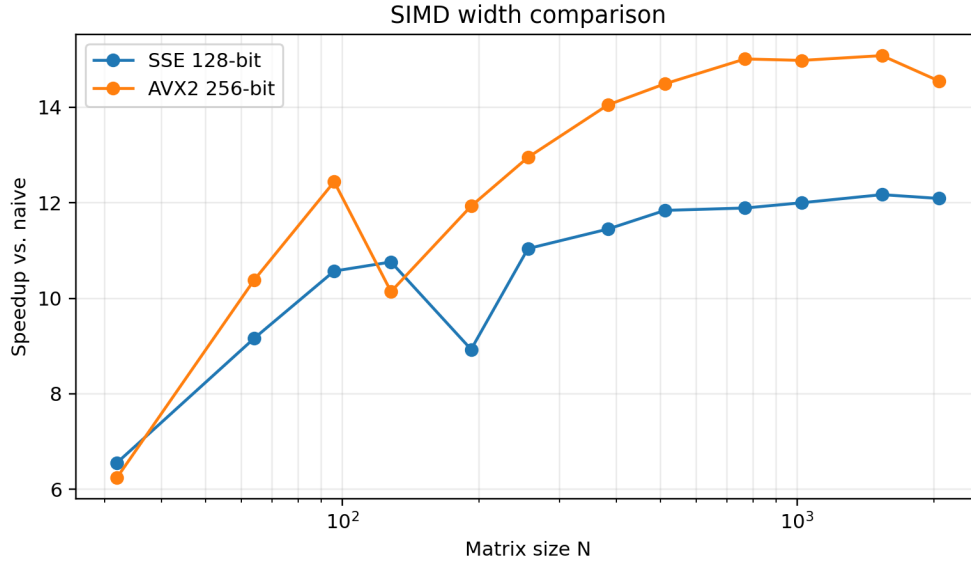
**Figure 3:** Five independent 256-bit AVX2 sweeps across matrix size. The kernel reaches a stable high-throughput region around  $N = 384$ – $1536$ , while the small and transition sizes show noticeably higher run-to-run variation. The approximate L1-D, L2 and LLC markers indicate aggregate working-set capacities rather than exact cache-transition points.

The five trials show the same broad behavior as the SSE experiment, but at a higher throughput level. Performance rises rapidly from the small-matrix region and reaches approximately 59 GFLOP/s around  $N = 384$ – $768$ . At the largest tested sizes, throughput decreases slightly as the working set grows beyond the private-cache capacities.

**Table 3:** Initial 256-bit AVX2 sweep across matrix size.

| $N$  | GFLOP/s | Speedup | Regime        |
|------|---------|---------|---------------|
| 32   | 15.93   | 6.25    | small         |
| 64   | 29.70   | 10.38   | small         |
| 96   | 20.06   | 12.44   | transition    |
| 128  | 48.64   | 10.14   | transition    |
| 192  | 57.20   | 11.93   | L1/L2         |
| 256  | 57.94   | 12.95   | L1/L2         |
| 384  | 59.10   | 14.05   | L2 transition |
| 512  | 59.40   | 14.49   | beyond L2     |
| 768  | 59.82   | 15.01   | beyond L2     |
| 1024 | 58.15   | 14.98   | beyond L2     |
| 1536 | 57.99   | 15.08   | beyond L2     |
| 2048 | 55.55   | 14.55   | beyond L2     |

The representative sweep reaches a peak of about 59.8 GFLOP/s at  $N = 768$ . The five-trial plot is useful here because individual points in the transition region can move substantially between runs; consequently, we use the broad steady-state trend rather than a single noisy point to judge the quality of the AVX2 kernel.



**Figure 4:** Matched SIMD-width comparison. AVX2 provides a clear steady-state improvement over 128-bit SSE, although the gain is well below the ideal factor of two from vector width alone.

For large matrices AVX2 reaches approximately 55–60 GFLOP/s, compared with 46–48 GFLOP/s for SSE. The improvement is therefore real but not  $2\times$ : loads, loop overhead, cache behavior and the repeated streaming of the unblocked operand remain common costs. The matched-width comparison also shows that the wider vector is not uniformly better at every individual size; for example, transition-region measurements can reverse the ordering because of run-to-run and cache effects.

AVX-512 was not supported by the measurement processor, so no 512-bit result is reported. A 64-bit FP32 SIMD path was also not used; the practical Intel SIMD paths available for this kernel were SSE and AVX2.

### 2.3 Instruction-count evidence

The assignment also asks for instruction-count analysis. The following `perf` measurements were collected at representative sizes. They count the complete benchmark process, including reference/correctness work, rather than an isolated kernel-only region.

**Table 4:** Representative `perf` measurements for naive and AVX2 runs.

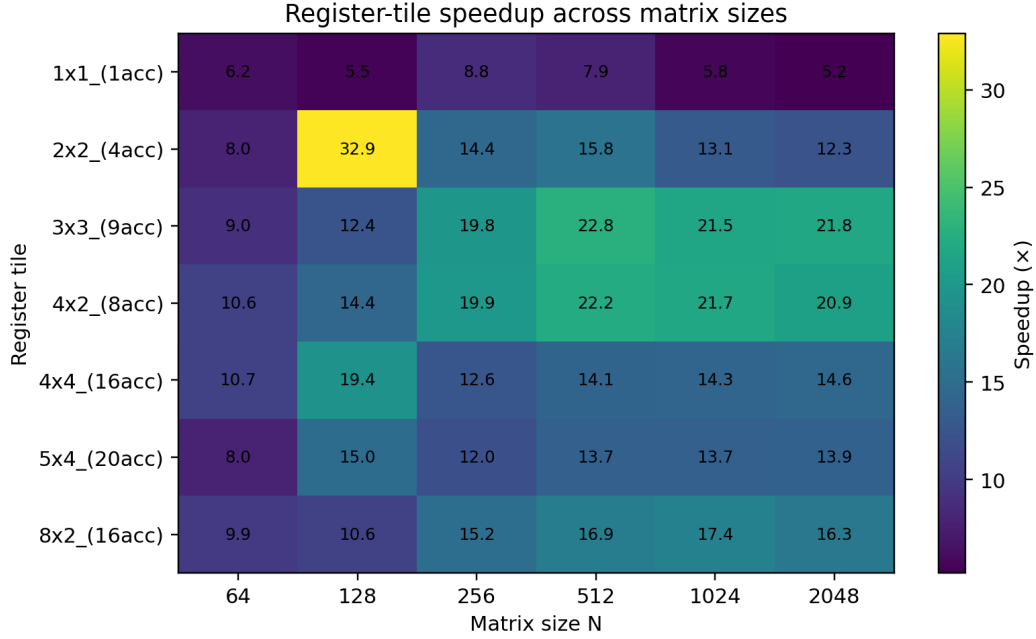
| Configuration     | Instructions           | Cycles                 | L1-D load misses    | LLC load misses |
|-------------------|------------------------|------------------------|---------------------|-----------------|
| Naive, $N = 512$  | $4.947 \times 10^9$    | $1.523 \times 10^9$    | $5.174 \times 10^6$ | 4,980           |
| AVX2, $N = 512$   | $5.157 \times 10^9$    | $1.609 \times 10^9$    | $8.157 \times 10^6$ | 1,524           |
| Naive, $N = 1024$ | $3.895 \times 10^{10}$ | $1.256 \times 10^{10}$ | $7.329 \times 10^7$ | 55,114          |
| AVX2, $N = 1024$  | $4.063 \times 10^{10}$ | $1.326 \times 10^{10}$ | $7.380 \times 10^7$ | 38,382          |

The instruction count should not be interpreted as kernel-only work because the benchmark harness contributes substantially. The useful signal is the much lower execution time together

with lower LLC load misses at  $N = 1024$ . This is consistent with the wider-vector kernel doing the same mathematical work more efficiently rather than simply executing fewer whole-process instructions.

## 2.4 Register-tile exploration

Vector width alone did not give the best available kernel. We therefore varied the output-register tile. An  $r \times c$  tile keeps  $rc$  independent vector accumulators, allowing multiple FMA dependency chains and reuse of loaded input vectors.



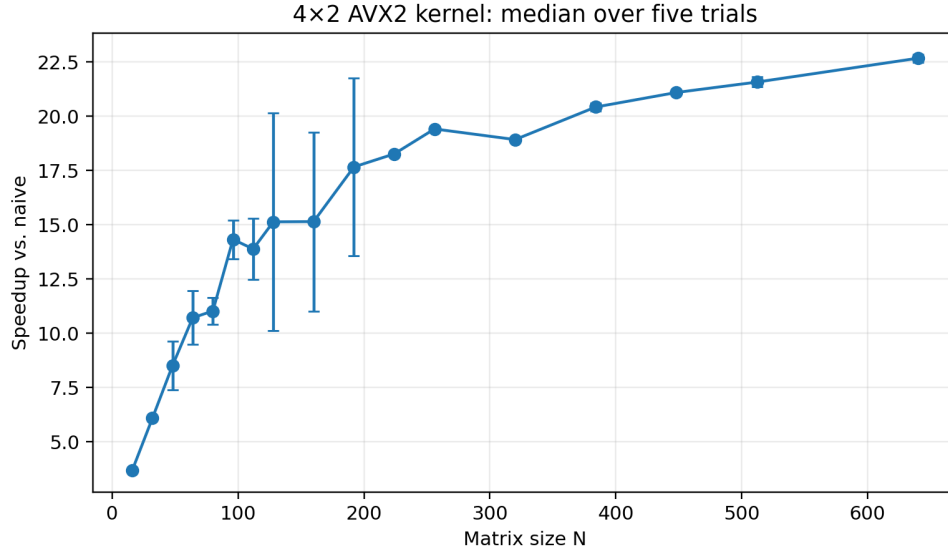
**Figure 5:** Register-tile speedup across matrix sizes. The  $4 \times 2$  and  $3 \times 3$  shapes are strong for large matrices, while wider accumulator sets regress.

**Table 5:** Register-tile sweep; entries are speedup over naive.

| Tile         | 64    | 128                | 256   | 512   | 1024  | 2048  |
|--------------|-------|--------------------|-------|-------|-------|-------|
| $1 \times 1$ | 6.19  | 5.52               | 8.75  | 7.90  | 5.75  | 5.21  |
| $2 \times 2$ | 8.02  | 32.89 <sup>†</sup> | 14.42 | 15.76 | 13.07 | 12.29 |
| $4 \times 2$ | 10.60 | 14.37              | 19.94 | 22.19 | 21.66 | 20.95 |
| $3 \times 3$ | 8.96  | 12.40              | 19.84 | 22.76 | 21.48 | 21.75 |
| $8 \times 2$ | 9.86  | 10.63              | 15.18 | 16.92 | 17.40 | 16.33 |
| $4 \times 4$ | 10.65 | 19.44              | 12.64 | 14.14 | 14.29 | 14.57 |
| $5 \times 4$ | 7.95  | 15.02              | 11.96 | 13.73 | 13.73 | 13.86 |

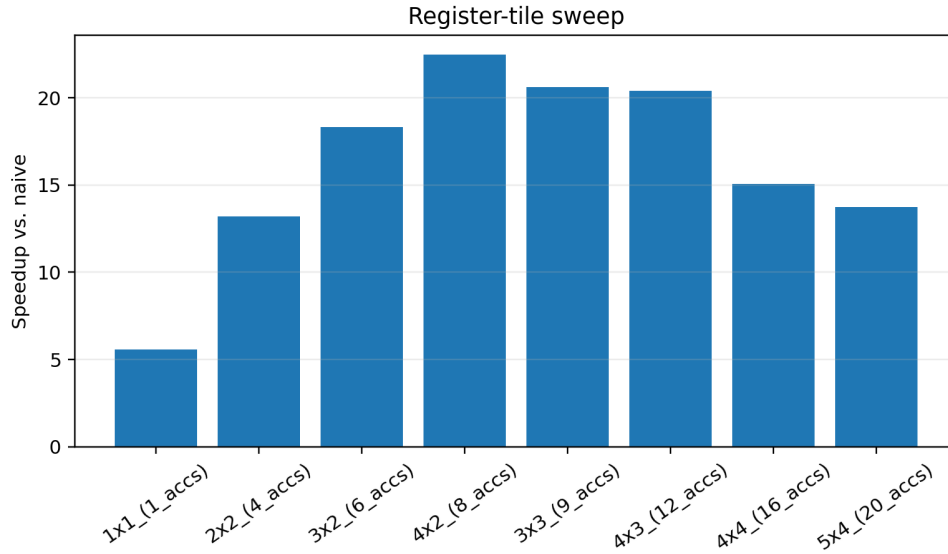
<sup>†</sup> Single-run anomaly at  $N = 128$ ; not used to select the final tile.

The  $4 \times 2$  and  $3 \times 3$  shapes are the strongest general-purpose choices from  $N = 256$  onward. Larger tiles do not automatically improve performance. Their additional live accumulators increase register pressure and can introduce extra register-management or spill traffic. Thus the number of accumulators has to be balanced against the available YMM register file and the live operand state.



**Figure 6:** Five-trial validation of the  $4 \times 2$  SIMD kernel. The large-matrix region is much more stable than the small-size transition region.

The repeated  $4 \times 2$  sweep reaches  $22.68\times$  speedup and 91.06 GFLOP/s at  $N = 640$ . Its standard deviation falls below 0.25 speedup points for  $N \geq 224$ , giving confidence that  $4 \times 2$  is a strong SIMD-only choice rather than a single noisy optimum.



**Figure 7:** Focused register-tille comparison. The  $4 \times 2$  tile gives the strongest SIMD-only result among the directly compared configurations.

## 2.5 SIMD conclusion

The SIMD stage therefore established two separate results. First, 256-bit AVX2 is the best supported vector width on the target CPU. Second, within AVX2, register geometry matters: the  $4 \times 2$  tile was selected as the SIMD-only micro-kernel. This kernel becomes the starting point for the memory-side optimization rather than the final combined kernel.



### 3 Task 2A — Software Prefetching

#### 3.1 Why cache blocking is added

After SIMD optimization, the arithmetic part of the inner product is substantially faster. The remaining bottleneck is increasingly the movement of the large operand. The software-prefetch stage therefore adds output-dimension blocking so that a smaller  $B$  panel is reused while many rows of  $A$  are processed.

For a  $B$  panel of width  $N_c$ ,

$$S_B \approx N_c K \times 4 \text{ bytes.} \quad (3)$$

For  $K = 2048$ ,  $N_c = 64, 128, 256, 512$  correspond to approximately 512 KB, 1 MB, 2 MB and 4 MB respectively. These are working-set guides rather than exact cache boundaries because associativity and other live data also matter.

#### 3.2 Cache-panel sweep

At  $N = 2048$  the panel sweep was:

**Table 6:** Cache-panel width sweep at  $N = 2048$ .

| $N_c$ | HW PF OFF     | HW PF ON      |
|-------|---------------|---------------|
| 64    | 23.08×        | 24.27×        |
| 128   | <b>23.43×</b> | <b>24.48×</b> |
| 256   | 22.68×        | 24.44×        |
| 512   | 21.44×        | 23.47×        |
| 1024  | 21.67×        | 23.49×        |

$N_c = 128$  is the strongest measured region under both hardware-prefetcher states. Increasing the panel to  $N_c = 512$  makes the panel about 4 MB for  $K = 2048$ , beyond the nominal private L2 capacity, and performance degrades. This supports the cache-working-set explanation without claiming a single exact cache-capacity threshold.

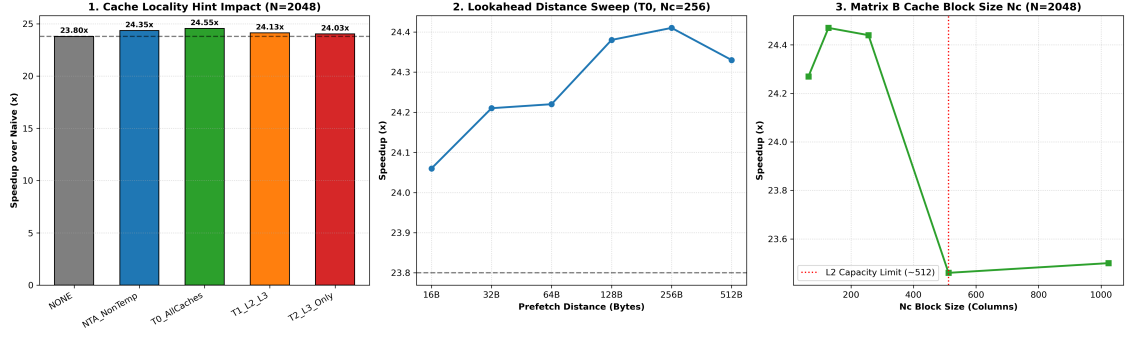
#### 3.3 Cache-fill level

The tested locality hints were `_MM_HINT_T0`, T1, T2 and NTA, with no explicit software prefetch as the baseline.

**Table 7:** Locality-hint sweep at  $N = 2048$ .

| Hint | HW PF OFF     | HW PF ON      |
|------|---------------|---------------|
| None | 22.64×        | 23.80×        |
| T0   | <b>22.79×</b> | <b>24.55×</b> |
| T1   | 22.09×        | 24.13×        |
| T2   | 22.07×        | 24.03×        |
| NTA  | 21.99×        | 24.35×        |

T0 is the best measured hint in both hardware-prefetcher states. The blocked  $B$  panel is reused across many rows of  $A$ , so bringing the prefetched data into the closest cache levels is useful. The NTA hint is less appropriate for this reuse pattern.



**Figure 8:** Software-prefetch parameter study at  $N = 2048$ . The three panels isolate the effects of locality hint, lookahead distance, and cache-panel width under hardware-prefetcher ON/OFF.

### 3.4 Prefetch distance

With T0 fixed, the lookahead distance was swept:

**Table 8:** Prefetch-distance sweep at  $N = 2048$ , T0 hint.

| Distance | HW PF OFF     | HW PF ON      |
|----------|---------------|---------------|
| 16 B     | 21.79×        | 24.06×        |
| 32 B     | 22.06×        | 24.21×        |
| 64 B     | <b>22.77×</b> | 24.22×        |
| 128 B    | 22.63×        | 24.38×        |
| 256 B    | 22.55×        | <b>24.41×</b> |
| 512 B    | 22.60×        | 24.33×        |

The best distance depends on hardware-prefetcher state: 64 B when the hardware prefetcher is off and 256 B when it is on. Too little lookahead gives insufficient latency-hiding time, whereas excessive lookahead can increase cache and prefetch-queue pressure. The shift in the optimum shows that software prefetching cannot be tuned independently of the hardware prefetcher.

### 3.5 Hardware-prefetcher effect

The controlled ON/OFF experiment was used to separate the hardware and software effects.

**Table 9:** Hardware/software prefetch interaction at  $N = 2048$ .

| Configuration       | Speedup | GFLOP/s |
|---------------------|---------|---------|
| No SW PF, HW PF OFF | 22.64×  | 84.52   |
| No SW PF, HW PF ON  | 23.80×  | 90.82   |
| T0 SW PF, HW PF OFF | 22.79×  | —       |
| T0 SW PF, HW PF ON  | 24.55×  | —       |

Enabling the hardware prefetcher improves the tiled baseline, and the best software-prefetch configuration performs best with hardware prefetching enabled. Thus the two mechanisms are complementary for this workload, although their useful software lookahead distance changes.

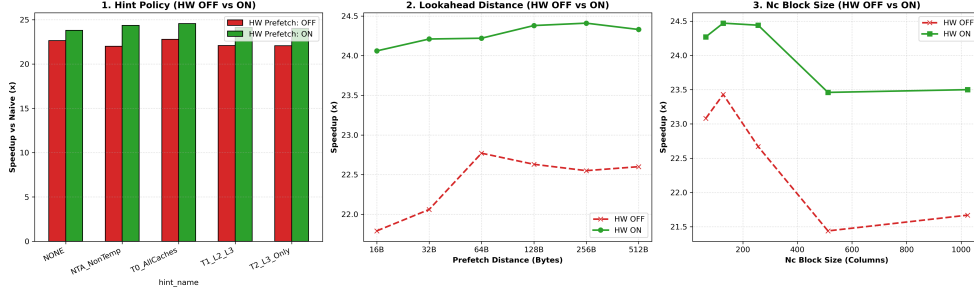


Figure 9: Hardware-prefetcher ON/OFF comparison.

### 3.6 Performance-counter coverage

The assignment requests L1-D, L2 and LLC miss trends as well as the `sw_prefetch_access` event. Reliable L1-D and LLC measurements were obtained for the representative SIMD campaign (Table 4). The available final prefetch measurement bundle did not contain a reliable L2-miss or `sw_prefetch_access` capture, so these values are deliberately not fabricated. The prefetch conclusions are therefore based on the controlled speedup sweeps, including hardware prefetcher isolation. This limitation is revisited in Section ??.

## 4 Task 2C — Combined SIMD + Software Prefetching

### 4.1 Combined autotuning

The SIMD-only winner was  $4 \times 2$ , but this does not imply that it remains optimal after changing the memory behavior of the kernel. Cache blocking and software prefetching alter the effective reuse and arrival time of the input data, so the register geometry was re-tuned with the selected prefetch configuration active.

The final combined campaign evaluated a substantially larger set of register-tile configurations with software prefetching enabled. The best configuration was a  $4 \times 3$  tile with software prefetching. This is an important distinction from the SIMD-only experiment:  $4 \times 2$  was the best isolated SIMD configuration, whereas  $4 \times 3$  was selected only after the memory-side optimization was introduced.

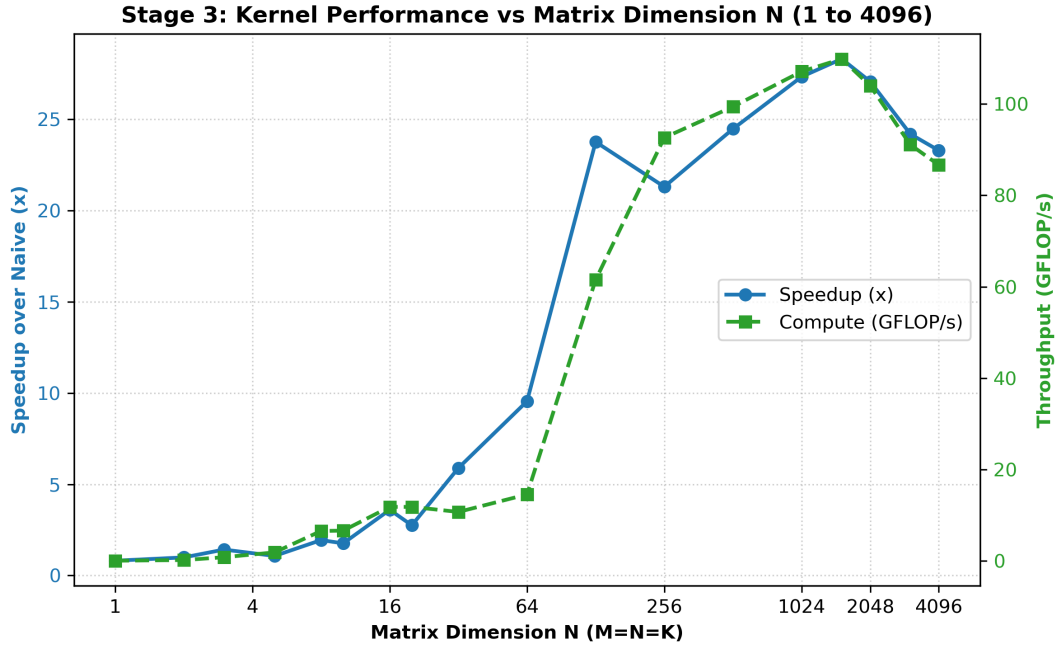
Table 10: Final optimization decisions from the staged experiments.

| Stage      | Parameter           | Selected configuration   |
|------------|---------------------|--------------------------|
| SIMD width | Vector width        | 256-bit AVX2             |
| SIMD-only  | Register tile       | $4 \times 2$             |
| Prefetch   | Locality hint       | <code>_MM_HINT_T0</code> |
| Prefetch   | Lookahead, HW PF ON | 256 B                    |
| Prefetch   | Cache-panel width   | $N_c = 128$              |
| Combined   | Register tile       | $4 \times 3$             |

The change from  $4 \times 2$  to  $4 \times 3$  demonstrates why the optimizations were tuned in stages. The additional accumulator state is not automatically beneficial in the pure SIMD kernel, but becomes useful when cache blocking and software prefetching have already reduced part of the memory-side bottleneck. Therefore, the final tile was chosen using the performance of the complete kernel rather than extrapolating from the SIMD-only sweep.

## 4.2 Final kernel performance across matrix size

After selecting the combined configuration, we evaluated the final kernel across a wider range of matrix dimensions. The plot below shows both speedup over the naive implementation and achieved compute throughput. The small-matrix region is dominated by fixed overheads, while performance increases rapidly as the matrix becomes large enough to amortize these costs. The kernel reaches approximately 100–110 GFLOP/s in the large-matrix region before throughput begins to decline for the largest matrices.



**Figure 10:** Final combined-kernel performance from  $N = 1$  to 4096. Speedup over the naive implementation and achieved throughput are shown together. The curves peak in the approximately  $N = 1024$ – $2048$  region before declining for the largest matrices.

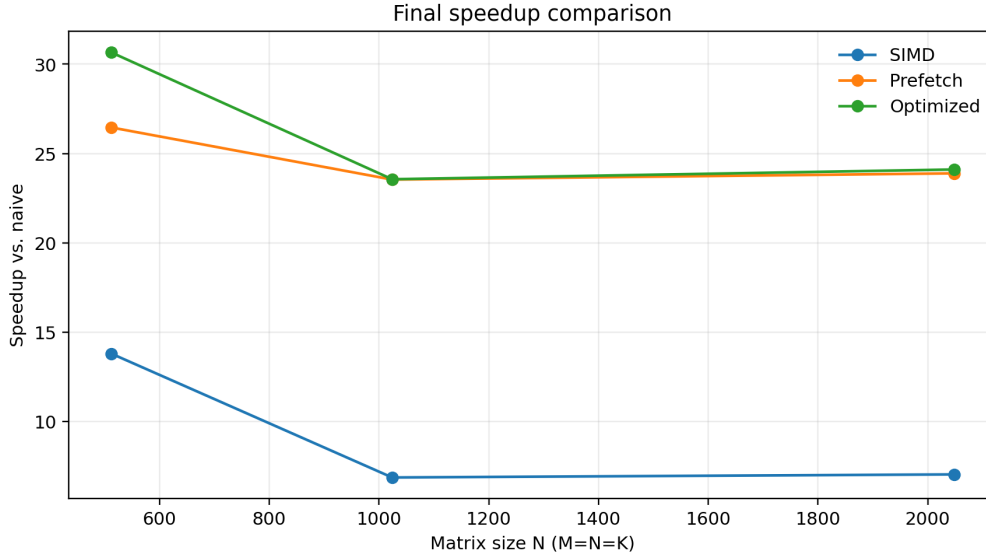
The scaling behavior is consistent with the changing balance between computation and memory traffic. At small  $N$ , the absolute workload is too small to fully amortize loop, blocking, and setup overheads. Throughput rises substantially once the matrices enter the steady-state region. Beyond the peak, increasing the working set further increases memory-system pressure, so the kernel no longer gains throughput from additional parallel work.

## 4.3 Final matched comparison

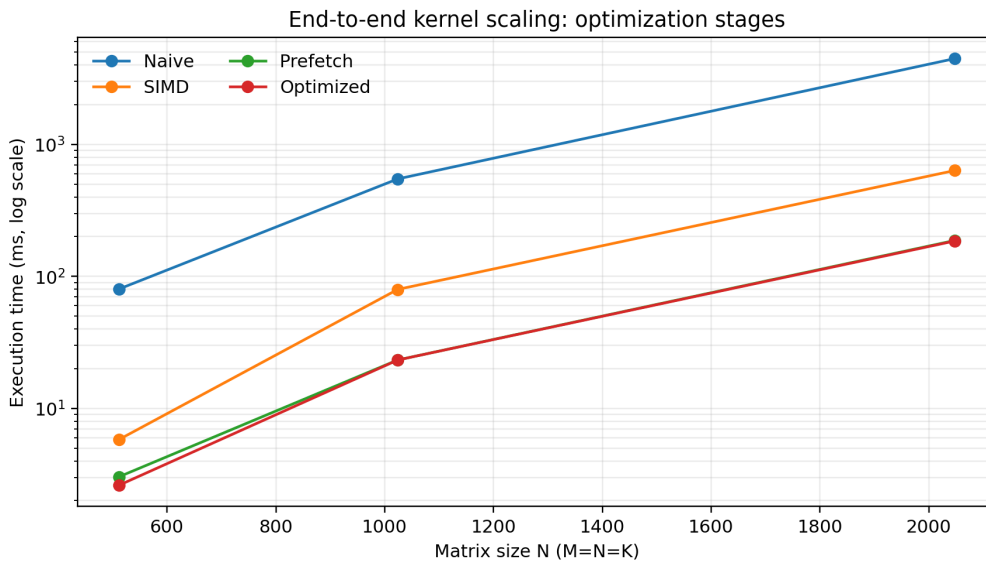
The following results come from the latest matched benchmark in which the four implementations were executed on the same workloads and executable build. This provides the direct comparison between the naive reference, SIMD-only kernel, prefetch-only kernel, and the final combined implementation.

**Table 11:** Final matched benchmark for square matrices.

| $N$  | Naive ms | SIMD ms | PF ms   | Opt. ms | SIMD   | PF     | Opt.          |
|------|----------|---------|---------|---------|--------|--------|---------------|
| 512  | 80.334   | 5.818   | 3.036   | 2.620   | 13.81× | 26.46× | <b>30.66×</b> |
| 1024 | 547.952  | 79.715  | 23.275  | 23.256  | 6.87×  | 23.54× | <b>23.56×</b> |
| 2048 | 4474.916 | 635.111 | 187.297 | 185.622 | 7.05×  | 23.89× | <b>24.11×</b> |



**Figure 11:** Final matched speedup comparison across matrix sizes. The combined kernel gives the highest measured speedup at all three representative sizes.



**Figure 12:** Final matched execution time on a logarithmic scale. The memory optimizations produce the dominant reduction in runtime for large matrices.

At  $N = 512$ , the combined kernel reaches  $30.66\times$  speedup and 102.46 GFLOP/s, improving on the prefetch-only result by approximately 15.9%. At  $N = 1024$ , the two implementations are almost identical in this matched build:  $23.54\times$  for prefetch-only versus  $23.56\times$  for the combined kernel. At  $N = 2048$ , the combined kernel reaches  $24.11\times$ , compared with  $23.89\times$  for prefetch-only, giving a smaller improvement of approximately 0.9%.

This shows that the two optimizations are not multiplicative. Software prefetching and cache blocking already remove much of the memory-side bottleneck, leaving less headroom for the SIMD micro-kernel to improve the final runtime. The benefit of the combined implementation is therefore largest where the compute and memory costs are more evenly balanced, rather than necessarily at the largest matrix size.

#### 4.4 Overall optimization effect

The staged experiments give a clear progression:

1. 128-bit SSE established the first major improvement, reaching approximately  $12\times$  speedup in the steady-state region.
2. 256-bit AVX2 increased the arithmetic throughput to approximately 55–60 GFLOP/s in the pure-vector kernel.
3. Register tiling increased the SIMD-only throughput substantially, with  $4 \times 2$  selected as the best isolated SIMD configuration.
4. Software prefetching and cache blocking provided the dominant improvement for the large-memory regime, reaching approximately 92 GFLOP/s in the matched comparison.
5. Re-tuning the register geometry with prefetching enabled selected  $4 \times 3$  as the final combined kernel, reaching 102.46 GFLOP/s at  $N = 512$ .

Thus, the final implementation was obtained by treating SIMD width, register reuse, cache blocking, and software prefetching as interacting parameters rather than independently maximizing each technique.

### 5 llama.cpp Integration

The final kernel was integrated into the pinned `llama.cpp` build. An adapter maps the ggml tensor layout and strides to the student SGEMM interface, and the relevant CPU F32 matrix multiplication path is redirected to `matmul_optimized`. The demonstration is single-threaded and uses a small F32 model.

Three independent runs were used for the final timing summary. The student implementation generated the same output as the stock implementation token-for-token, and the integration log confirmed that `matmul_optimized` was invoked.

**Table 12:** llama.cpp end-to-end comparison over three runs; medians are reported.

| Metric          | Stock         | Student       | Ratio                |
|-----------------|---------------|---------------|----------------------|
| Eval time       | 37.02 ms      | 6.97 ms       | $5.31\times$         |
| Eval throughput | 1701.97 tok/s | 9043.93 tok/s | $5.31\times$         |
| Prompt time     | 1.64 ms       | 0.28 ms       | $\approx 5.66\times$ |

The evaluation result is the more meaningful end-to-end metric: the optimized student SGEMM improves measured single-threaded evaluation throughput by approximately  $5.3\times$  while preserving the generated output exactly.

### 6 Conclusion

The optimization followed the hardware bottleneck as it changed. SIMD first removed much of the scalar arithmetic cost, with 256-bit AVX2 clearly outperforming SSE. Register blocking then exposed more independent FMA chains, with  $4 \times 2$  emerging as the strongest SIMD-only geometry. Once the kernel became limited by data movement, cache blocking and software

prefetching became the dominant optimization:  $T_0$ ,  $N_c = 128$ , and a 256 B lookahead with hardware prefetching enabled formed the strongest measured prefetch configuration. Finally, a larger combined autotuning search selected a  $4 \times 3$  register tile, showing that the best micro-kernel in isolation need not be the best micro-kernel when memory behavior is included.

The final matched implementation reaches more than  $30\times$  speedup over the supplied naive kernel and was validated inside `llama.cpp`. The student kernel was actually invoked, its output matched the stock implementation token-for-token, and the measured single-threaded inference evaluation throughput improved by approximately  $5.3\times$ .

## Code Repository

The complete source code for this assignment is available on GitHub:

[https://github.com/Urjith-K/cs683\\_assignment\\_1](https://github.com/Urjith-K/cs683_assignment_1)